Unit 2 – Process Synchronization

On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process**: The execution of one process does not affect the execution of other processes.
- **Cooperative Process**: A process that can affect or be affected by other processes executing in the system.

Process synchronization problem arises in the case of Cooperative process also because resources are shared in Cooperative processes.

Race Condition:

A race condition is **an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time**,

Critical Section Problem:

A critical section is a code segment that can be accessed by only one process at a time. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (TRUE);
```

In the entry section, the process requests for entry in the **Critical Section.**
Any solution to the critical section problem must satisfy three requirements:

- **Mutual Exclusion**: If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
- **Progress**: If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter in the critical section next, and the selection can not be postponed indefinitely.
- **Bounded Waiting**: A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's Solution:

Peterson's Solution is a classical software-based solution to the critical section problem. In Peterson's solution, we have two shared variables:

- boolean flag[i]: Initialized to FALSE, initially no one is interested in entering the critical section
- int turn: The process whose turn is to enter the critical section.

```
do {

    flag[i] = TRUE ;
    turn = j ;
    while (flag[j]  &&  turn == j) ;

        critial section

    flag[i] = FALSE ;

        remainder section

  }  while (TRUE) ;
```

Peterson's Solution preserves all three conditions:

- Mutual Exclusion is assured as only one process can access the critical section at any time.
- Progress is also assured, as a process outside the critical section does not block other processes from entering the critical section.
- Bounded Waiting is preserved as every process gets a fair chance.

Disadvantages of Peterson's solution:

- It involves busy waiting.(In the Peterson's solution, the code statement- "while(flag[j] && turn == j);" is responsible for this. Busy waiting is not favored because it wastes CPU cycles that could be used to perform other tasks.)
- It is limited to 2 processes.
- Peterson's solution cannot be used in modern CPU architectures.

Semaphores:

A semaphore is a signaling mechanism and a thread that is waiting on a semaphore can be signaled by another thread. This is different than a mutex as the mutex can be signaled only by the thread that is called the wait function.

A semaphore uses two atomic operations, wait and signal for process synchronization.
A Semaphore is an integer variable, which can be accessed only through two operations wait() and signal().

wait operation()

```
Copy CodeP(S)
{
    while (S<=0);
    S--;
}
```

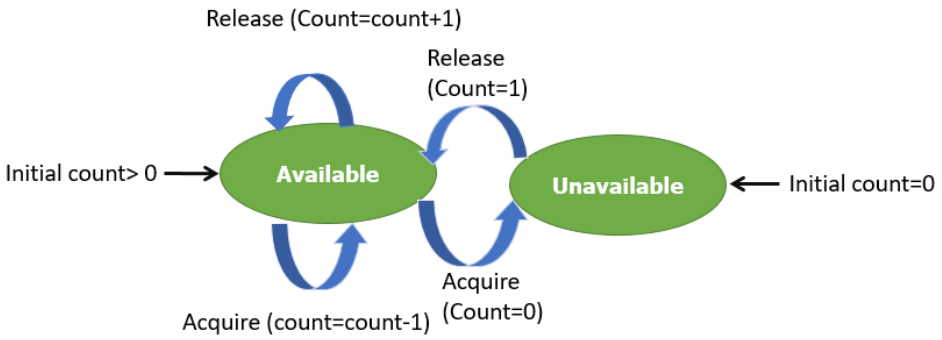
Signal operation()

```
Copy CodeP(S)
{
    while (S>=0);
    S++;
}
```

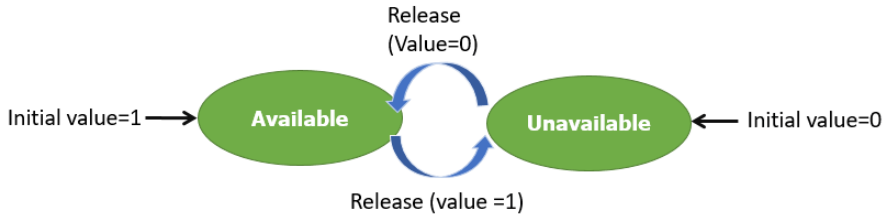
There are two types of semaphores: Binary Semaphores and Counting Semaphores.

- **Binary Semaphores:** They can only be either 0 or 1. They are also known as mutex locks, as the locks can provide mutual exclusion. All the processes can share the same mutex semaphore that is initialized to 1. Then, a process has to wait until the lock becomes 0. Then, the process can make the mutex semaphore 1 and start its critical section. When it completes its critical section, it can reset the

value of the mutex semaphore to 0 and some other process can enter its critical section.



- **Counting Semaphores:** They can have any value and are not restricted over a certain domain. They can be used to control access to a resource that has a limitation on the number of simultaneous accesses. The semaphore can be initialized to the number of instances of the resource. Whenever a process wants to use that resource, it checks if the number of remaining instances is more than zero, i.e., the process has an instance available. Then, the process can enter its critical section thereby decreasing the value of the counting semaphore by 1. After the process is over with the use of the instance of the resource, it can leave the critical section thereby adding 1 to the number of available instances of the resource.



Advantages of Semaphores

Here, are pros/benefits of using Semaphore:

- It allows more than one thread to access the critical section
- Semaphores are machine-independent.
- They do not allow multiple processes to enter the critical section.
- As there is busy waiting in semaphore, there is never a wastage of process time and resources.
- They are machine-independent, which should be run in the machine-independent code of the microkernel.
- They allow flexible management of resources.

Disadvantage of semaphores

Here, are cons/drawback of semaphore

- One of the biggest limitations of a semaphore is priority inversion.
- The operating system has to keep track of all calls to wait and signal semaphore.
- In order to avoid deadlocks in semaphore, the Wait and Signal operations require to be executed in the correct order.
- Semaphore programming is a complicated, so there are chances of not achieving mutual exclusion.

Reference:
https://www.geeksforgeeks.org/introduction-of-process-synchronization/
https://www.guru99.com/semaphore-in-operating-system.html

Hardware Based Solution to Synchronization:

There is no guarantee that the Software-based solution like Peterson's will work on modern architecture. Hence, certain hardware-based solution to synchronization are proposed. Two such solutions are:

1. TestAndSet() instruction
2. Swap() instruction

3. Unlock and Lock

Both these instructions are ***atomic*** instructions which means that when a process is executing any of these instructions it can not be preempted until the instruction is complete.

1. TestAndSet() Instructions

TestAndSet() instruction uses a boolean variable ***lock.*** The initial value of the *lock* is false. The variable lock ensures mutual exclusion. If the value of *lock* is false, this means that no process is in its critical section. Hence, the value *true* means that some process is running in its critical section.

Definition of TestAndSet() Instruction

```
boolean
TestAndSet(boolean *lock)
{
boolean initial = *lock;
*lock = TRUE;
return initial ;
}
```

Implementation of TestAndSet() Instruction

```
do {
  while
(TestAndSet(&lock));
  critical section
lock = FALSE;
  remainder section
} while(TRUE);
```

2. Swap() Instruction

The swap() instruction uses two boolean variables *lock* and *key*.

Definition of Swap() Instruction

```
void Swap(boolean *a,
boolean *b)
{
Boolean t = *a;
*a = *b;
*b = t;
}
```

Implementation of Swap() Instruction

```
do {
key = TRUE;
while (key == TRUE)
     Swap(&lock, &key);
//critical section

lock = FALSE;

//remainder section
} while (TRUE);
```

Lock and unlock:

Unlock and Lock

In addition to Test and Set, this algorithm uses waiting[i] to check if there are any processes in the wait. The processes are set in the ready queue with respect to the critical section.

```
do {
  acquire lock
     critical section
  release lock
     remainder section
} while (TRUE);
```

Advantages and Disadvantages of Synchronization Hardware

Advantages of Hardware Instruction

- Hardware instructions are easy to implement and improves the efficiency of the system.
- Supports any number of processes may it be on the single or multiple processor system.
- With hardware instructions, you can implement multiple critical sections each defined with a unique variable.

Disadvantages of Hardware Instruction

- Processes waiting for entering their critical section consumes a lot of processors time which increases busy waiting.
- As the selection of processes to enter their critical section is arbitrary. It may happen that some processes are waiting for the indefinite time which leads to process starvation.
- Deadlock is also possible.

Advantages and Disadvantages of Synchronization Hardware

Advantages of Hardware Instruction

- Hardware instructions are easy to implement and improves the efficiency of the system.

- Supports any number of processes may it be on the single or multiple processor system.
- With hardware instructions, you can implement multiple critical sections each defined with a unique variable.

Disadvantages of Hardware Instruction

- Processes waiting for entering their critical section consumes a lot of processors time which increases busy waiting.
- As the selection of processes to enter their critical section is arbitrary. It may happen that some processes are waiting for the indefinite time which leads to process starvation.
- Deadlock is also possible.

Classical problems of Synchronization with Semaphore Solution

The following problems of synchronization are considered as classical problems:

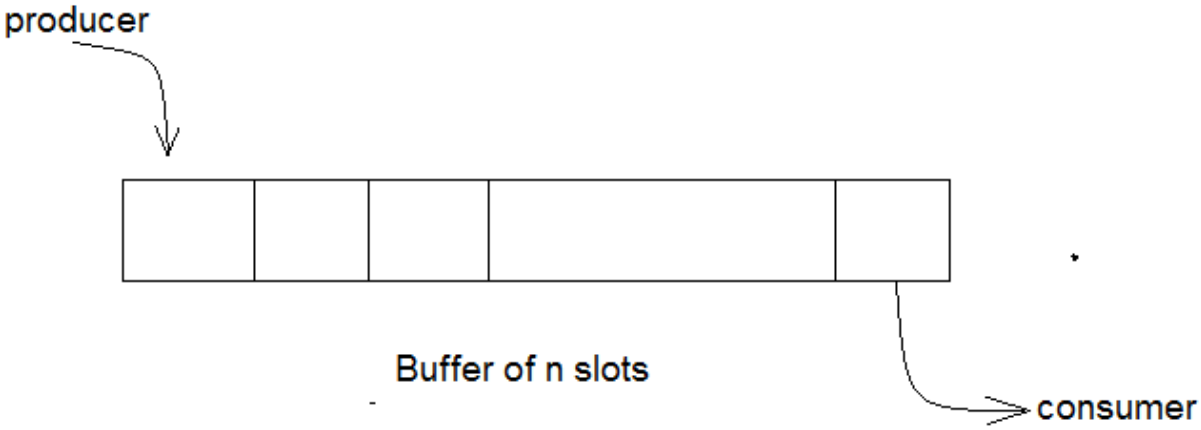
1. Bounded-buffer (or Producer-Consumer) Problem,
2. Dining-Philosphers Problem,
3. Readers and Writers Problem,

BOUNDED BUFFER PROBLEM

Bounded buffer problem, which is also called **producer consumer problem**, is one of the classic problems of synchronization.

Problem Statement:

There is a buffer of **n** slots and each slot is capable of storing one unit of data. There are two processes running, namely, **producer** and **consumer**, which are operating on the buffer.



Bounded Buffer Problem

A producer tries to insert data into an empty slot of the buffer. A consumer tries to remove data from a filled slot in the buffer. As you might have guessed by now, those two processes won't produce the expected output if they are being executed concurrently.

There needs to be a way to make the producer and consumer work in an independent manner.

Solution:

One solution of this problem is to use semaphores. The semaphores which will be used here are:

- **m**, a binary semaphore which is used to acquire and release the lock.
- **empty**, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
- **full**, a counting semaphore whose initial value is 0.

At any instant, the current value of empty represents the number of empty slots in the buffer and full represents the number of occupied slots in the buffer.

Producer Operation:

The pseudocode of the producer function looks like this:

```
do {
  wait(empty); // wait until empty>0 and then decrement 'empty'
  wait(mutex); // acquire lock
  /* perform the
  insert operation in a slot */
  signal(mutex); // release lock
  signal(full);  // increment 'full'
} while(TRUE)
```

- Looking at the above code for a producer, we can see that a producer first waits until there is atleast one empty slot.
- Then it decrements the **empty** semaphore because, there will now be one less empty slot, since the producer is going to insert data in one of those slots.
- Then, it acquires lock on the buffer, so that the consumer cannot access the buffer until producer completes its operation.
- After performing the insert operation, the lock is released and the value of **full** is incremented because the producer has just filled a slot in the buffer.

Consumer Operation:

The pseudocode of the consumer function looks like this:

```
do {
```

```
  wait(full); // wait until full>0 and then decrement 'full'
  wait(mutex);  // acquire the lock
  /* perform the remove operation
   in a slot */
  signal(mutex); // release the lock
  signal(empty); // increment 'empty'
} while(TRUE);
```

- The consumer waits until there is atleast one full slot in the buffer.
- Then it decrements the **full** semaphore because the number of occupied slots will be decreased by one, after the consumer completes its operation.
- After that, the consumer acquires lock on the buffer.
- Following that, the consumer completes the removal operation so that the data from one of the full slots is removed.
- Then, the consumer releases the lock.
- Finally, the **empty** semaphore is incremented by 1, because the consumer has just removed data from an occupied slot, thus making it empty.

READERS WRITER PROBLEM:

Readers writer problem is another example of a classic synchronization problem. There are many variants of this problem, one of which is examined below.

The Problem Statement

There is a shared resource which should be accessed by multiple processes. There are two types of processes in this context. They are **reader** and **writer**. Any number of **readers** can read from the shared resource simultaneously, but only one **writer** can write to the shared resource. When a **writer** is writing data to the resource, no other process can access the resource. A **writer** cannot write to the resource if there are non zero number of readers accessing the resource at that time.

The Solution

From the above problem statement, it is evident that readers have higher priority than writer. If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.

Here, we use one **mutex** m and a **semaphore** w. An integer variable read_count is used to maintain the number of readers currently accessing the resource. The variable read_count is initialized to 0. A value of 1 is given initially to m and w.

Instead of having the process to acquire lock on the shared resource, we use the mutex m to make the process to acquire and release lock whenever it is updating the read_count variable.

The code for the **writer** process looks like this:

```
while(TRUE)
{
    wait(w);

   /* perform the write operation */

   signal(w);
}
```

And, the code for the **reader** process looks like this:

```
while(TRUE)
{
    //acquire lock
    wait(m);
    read_count++;
    if(read_count == 1)
        wait(w);

    //release lock
    signal(m);

    /* perform the reading operation */

    // acquire lock
    wait(m);
```

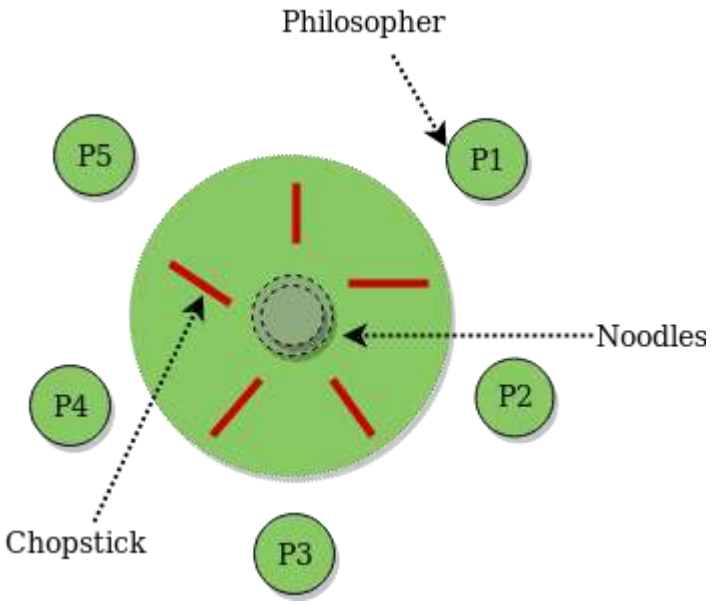
```
    read_count--;
    if(read_count == 0)
        signal(w);

    // release lock
    signal(m);
}
```

Explaination:

- As seen above in the code for the writer, the writer just waits on the **w** semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments **w** so that the next writer can access the resource.
- On the other hand, in the code for the reader, the lock is acquired whenever the **read_count** is updated by a process.
- When a reader wants to access the resource, first it increments the **read_count** value, then accesses the resource and then decrements the **read_count** value.
- The semaphore **w** is used by the first reader which enters the critical section and the last reader which exits the critical section.
- The reason for this is, when the first readers enters the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the **w** semaphore because there are zero readers now and a writer can have the chance to access the resource.

DINING PHILOSOPHERS PROBLEM (DPP)

The dining philosophers problem states that there are 5 philosophers sharing a circular table and they eat and think alternatively. There is a bowl of rice for each of the philosophers and 5 chopsticks. A philosopher needs both their right and left chopstick to eat. A hungry philosopher may only eat if there are both chopsticks available.Otherwise a philosopher puts down their chopstick and begin thinking again.



The dining philosopher is a classic synchronization problem as it demonstrates a large class of concurrency control problems.

Solution of Dining Philosophers Problem

A solution of the Dining Philosophers Problem is to use a semaphore to represent a chopstick. A chopstick can be picked up by executing a wait operation on the semaphore and released by executing a signal semaphore.

The structure of the chopstick is shown below:

```
semaphore chopstick [5];
```

Initially the elements of the chopstick are initialized to 1 as the chopsticks are on the table and not picked up by a philosopher.

The structure of a random philosopher i is given as follows:

```
do {

    wait( chopstick[i] );

    wait( chopstick[ (i+1) % 5] );

    .      .

    .  EATING THE RICE
```

```
    .
    signal( chopstick[i] );
    signal( chopstick[ (i+1) % 5] );
    .
    . THINKING
    .
} while(1);
```

In the above structure, first wait operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has picked up the chopsticks on his sides. Then the eating function is performed.

After that, signal operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has eaten and put down the chopsticks on his sides. Then the philosopher goes back to thinking.

Difficulty with the solution

The above solution makes sure that no two neighboring philosophers can eat at the same time. But this solution can lead to a deadlock. This may happen if all the philosophers pick their left chopstick simultaneously. Then none of them can eat and deadlock occurs.

Some of the ways to avoid deadlock are as follows:

- There should be at most four philosophers on the table.
- An even philosopher should pick the right chopstick and then the left chopstick while an odd philosopher should pick the left chopstick and then the right chopstick.
- A philosopher should only be allowed to pick their chopstick if both are available at the same time.

Monitors in Process Synchronization

The monitor is one of the ways to achieve Process synchronization. The monitor is supported by programming languages to achieve mutual exclusion between processes. For example Java Synchronized methods. Java provides wait() and notify() constructs.

1. It is the collection of condition variables and procedures combined together in a special kind of module or a package.
2. The processes running outside the monitor can't access the internal variable of the monitor but can call procedures of the monitor.
3. Only one process at a time can execute code inside monitors.

Syntax:

```
Monitor Demo //Name of Monitor
{
variables;
condition variables;

procedure p1 {....}
prodecure p2 {....}


}
```

Syntax of Monitor

Condition Variables:
Two different operations are performed on the condition variables of the monitor.

```
Wait.

signal.
```

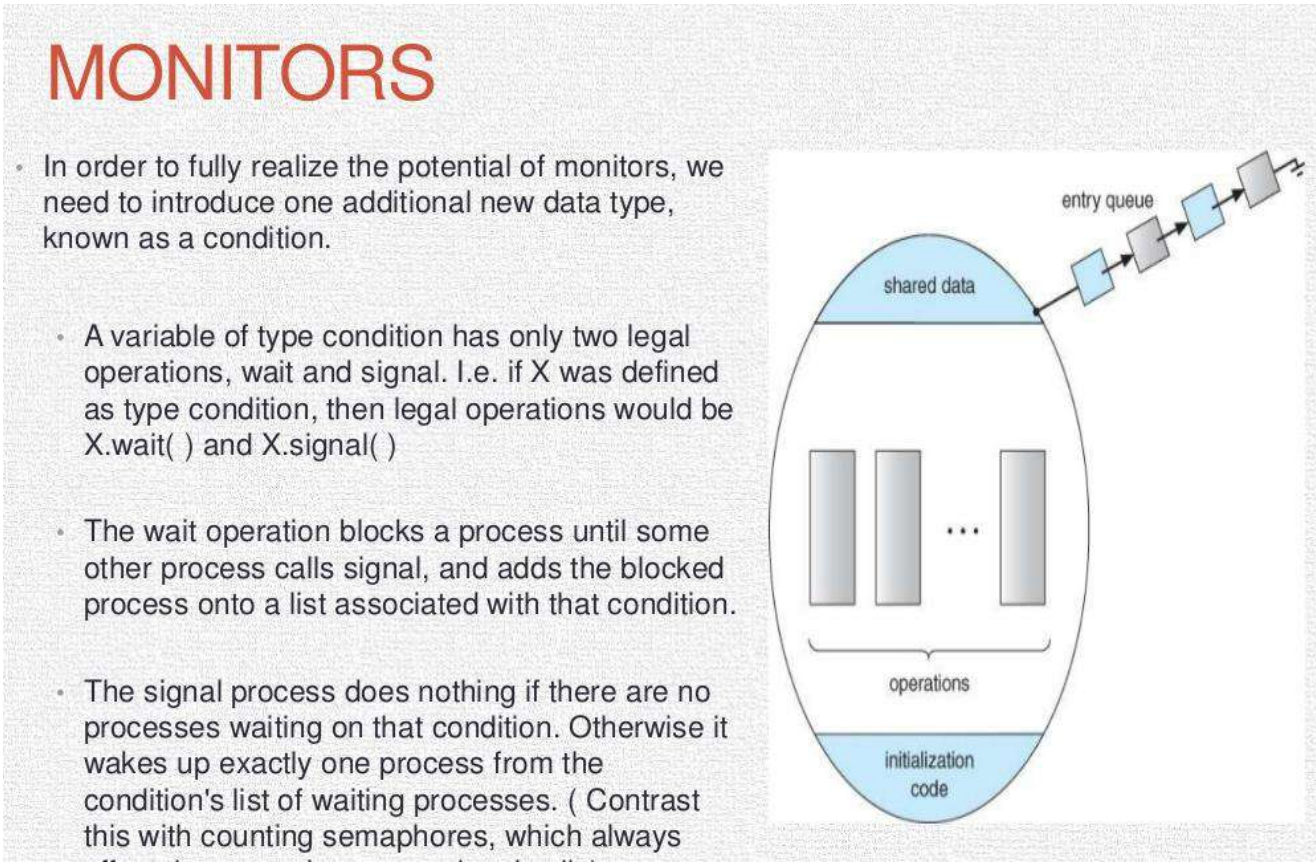
let say we have 2 condition variables
condition x, y; // Declaring variable

Wait operation
x.wait() : Process performing wait operation on any condition variable are suspended. The suspended processes are placed in block queue of that condition variable.
Note: Each condition variable has its unique block queue.

Signal operation
x.signal(): When a process performs signal operation on condition variable, one of the blocked processes is given chance.

```
If (x block queue empty)

  // Ignore signal
```

```
else
  // Resume a process from block queue.
```



Advantages of Monitor:
Monitors have the advantage of making parallel programming easier and less error prone than using techniques such as semaphore.
Disadvantages of Monitor:

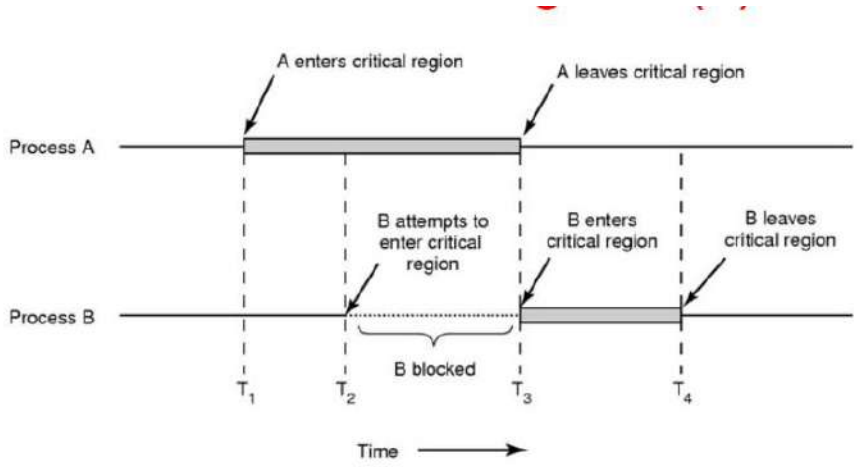
- Monitors have to be implemented as part of the programming language . The compiler must generate code for them.
- This gives the compiler the additional burden of having to know what operating system facilities are available to control access to critical sections in concurrent processes.
- Some languages that do support monitors are Java,C#,Visual Basic,Ada and concurrent Euclid.

CRITICAL REGION:

- A Critical Region is a section of code that is always executed under mutual exclusion.
- Critical regions shift the responsibility for enforcing mutual exclusion from the programmer to the compiler.
- They consist of two parts:
  - A variable that must be accessed under mutual exclusion
  - A new language statement that identifies a critical region in which the variable is accessed.

Four conditions to provide mutual exclusion

1. No two processes simultaneously in critical region
2. No assumptions made about speeds or numbers of CPUs
3. No process running outside its critical region may block another process
4. No process must wait forever to enter its critical region



Mutual exclusion using critical regions

If two processes A and B are in ready queue. If CPU is executing process A and process B in wait state. < If an interrupt occurs for process A, The operating system suspends the execution

of the first process, and store the current information of process A in its PCB and context to the second process namely process B.

In doing So, program counter from PCB of process B is loaded and the execution can continue with the new process.

Syllabus::

Process Synchronization and deadlocks: The Critical Section Problem, Synchronization hardware, Semaphores, Classical problems of synchronization, Critical regions, monitors.

UNIVERISTY QUESTIONS BASED ON ABOVE SYLLABUS:

Section A:

1. What is semaphores? Metion any two types of semaphores
2. What is race condition?
3. What is monitor?
4. What is critical section? Explain
5. Explain swap() operation in process synchronization.
6. **Difference between binary and counting semaphores?**
7. **What is mutual exclution**

Section B

1. **Write a short note on critical region**
2. **What is semaphores? Explain any two types of semaphores**
3. **Explain dining philosopher's problem in synchronization**
4. **Define critical section problem? What are the requirements for critical section problem**
5. **Explain any one classical problem of synchronization**
6. **Write a brief note on Peterson's algorithm**
7. **What is monitor? Explain in detail.**

DEADLOCKS

UNIT 2

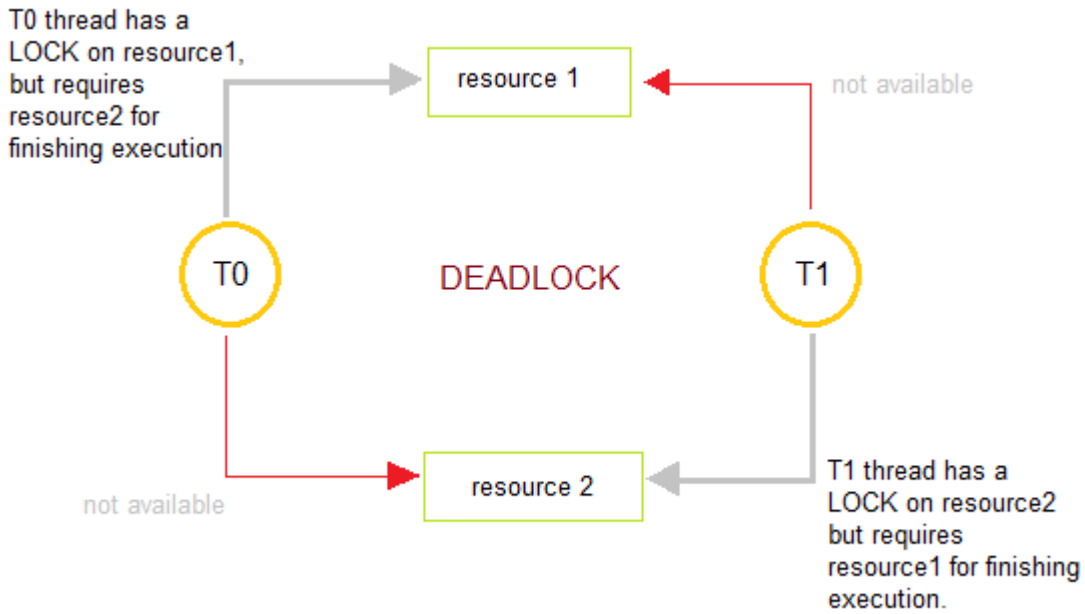
HAMELA K
DEPARTMENT OF COMPUTER SCIENCE
GOVERNMENT FIRST GRADE COLLEGE
MALUR 563 160

UNIT 2 - SYLLABUS

- Process Synchronization and deadlocks:
  - The Critical Section Problem
  - Synchronization hardware
  - Semaphores
  - Classical problems of synchronization
  - Critical regions
  - Monitors
- Dead locks :
  - Characterization
  - Dead lock prevention
  - avoidance and detection
  - Recovery from dead lock
  - Combined approach to deadlock handling.

DEADLOCKS

- Deadlocks are a set of blocked processes each holding a resource and waiting to acquire a resource held by another process.
-



- A process in operating systems uses different resources and uses resources in following way.
  1) Requests a resource
  2) Use the resource
  2) Releases the resource

DEADLOCK CHARACTERIZATION

- Necessary condition
- Resource-allocation graph

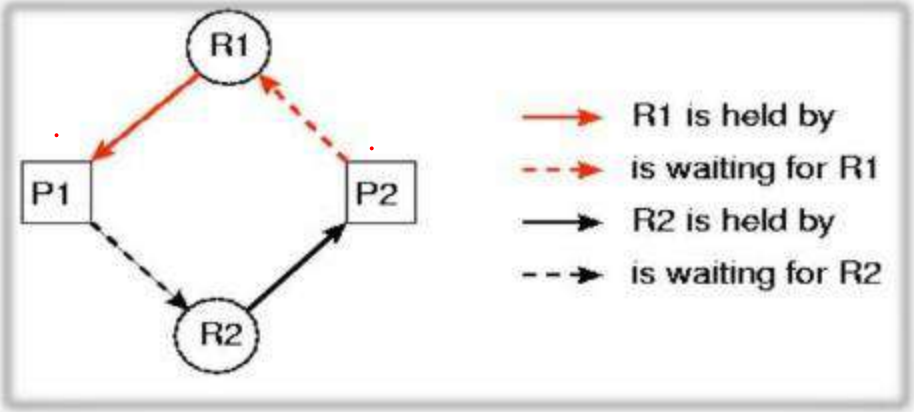
$P_1 \to R \to P_2$

NECESSARY CONDITION

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_0\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

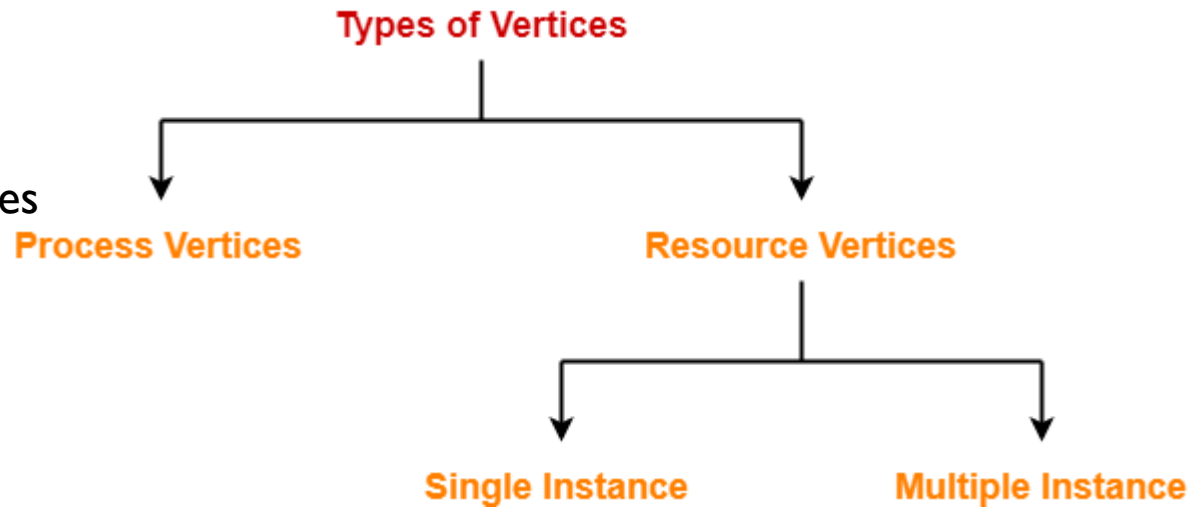
Circular wait



It only happens when there is a processes which are waiting for other resources to finish.

RESOURCE ALLOCATION GRAPH

- Resource Allocation Graph (RAG) is a graph that represents the state of a Directed Graph.
- **Components Of RAG-**
- There are two major components of a Resource Allocation Graph-
- Vertices
- Edges



Vertices-

There are following types of vertices in a Resource Allocation Graph-

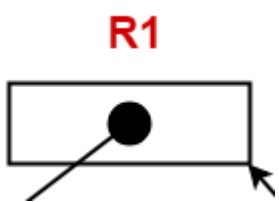
Process Vertices
Resource Vertices

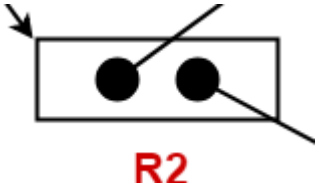
Process Vertices-

Process vertices represent the processes.
They are drawn as a circle by mentioning the name of process inside the circle.

Resource Vertices-

Resource vertices represent the resources.
Depending on the number of instances that exists in the system, resource vertices may be single instance or multiple instance.
They are drawn as a rectangle by mentioning the dots inside the rectangle.
The number of dots inside the rectangle indicates the number of instances of that resource existing in the system.

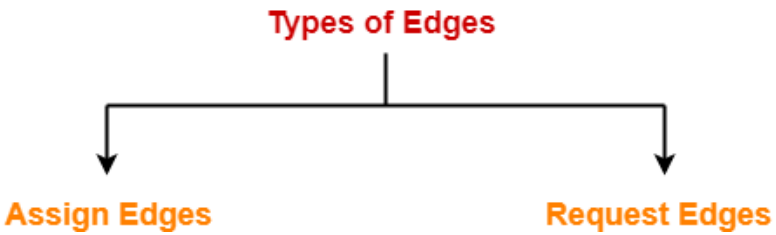




Edges-

There are two types of edges in a Resource Allocation Graph-

Assign Edges
Request Edges



Assign Edges-

Assign edges represent the assignment of resources to the processes.
They are drawn as an arrow where the head of the arrow points to the process and tail of the process points to the instance of the resource.

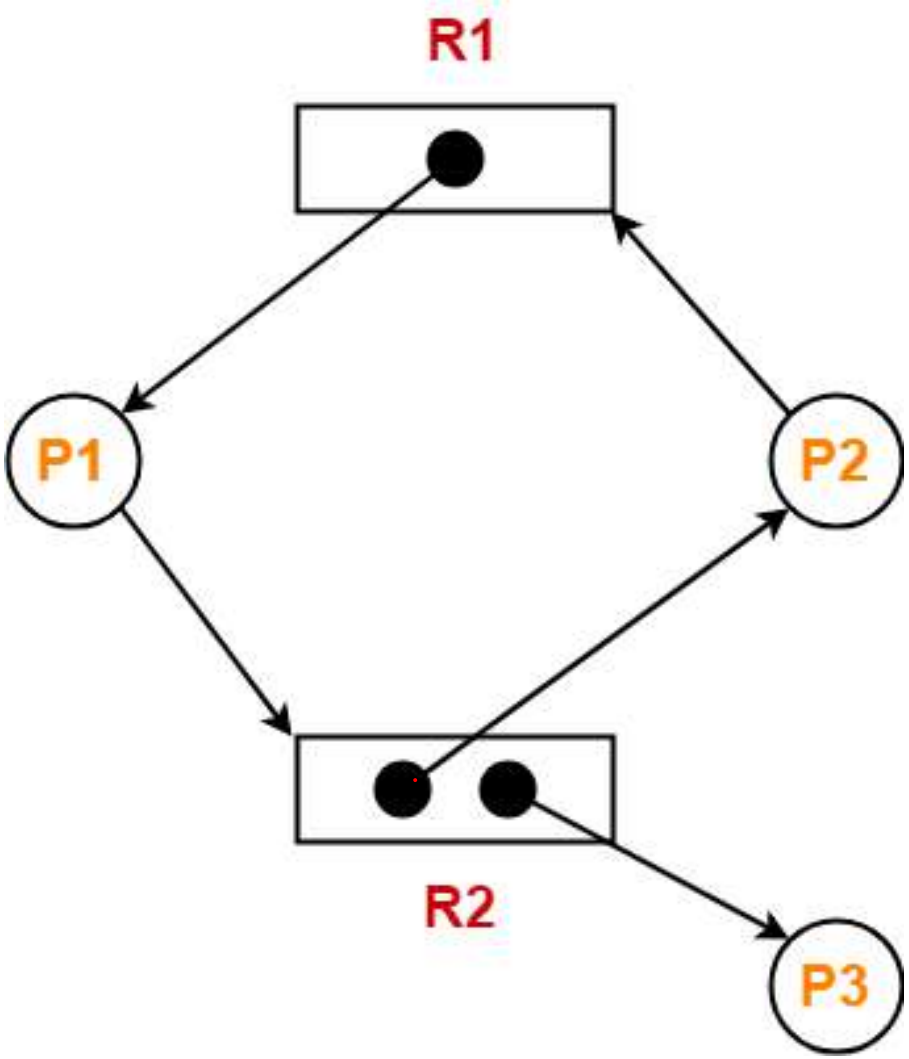
Request Edges-

Request edges represent the waiting state of processes for the resources.
They are drawn as an arrow where the head of the arrow points to the instance of the resource and tail of the process points to the process.
If a process requires 'n' instances of a resource type, then 'n' assign edges will be drawn.

EXAMPLE FOR RESOURCE ALLOCATION GRAPH

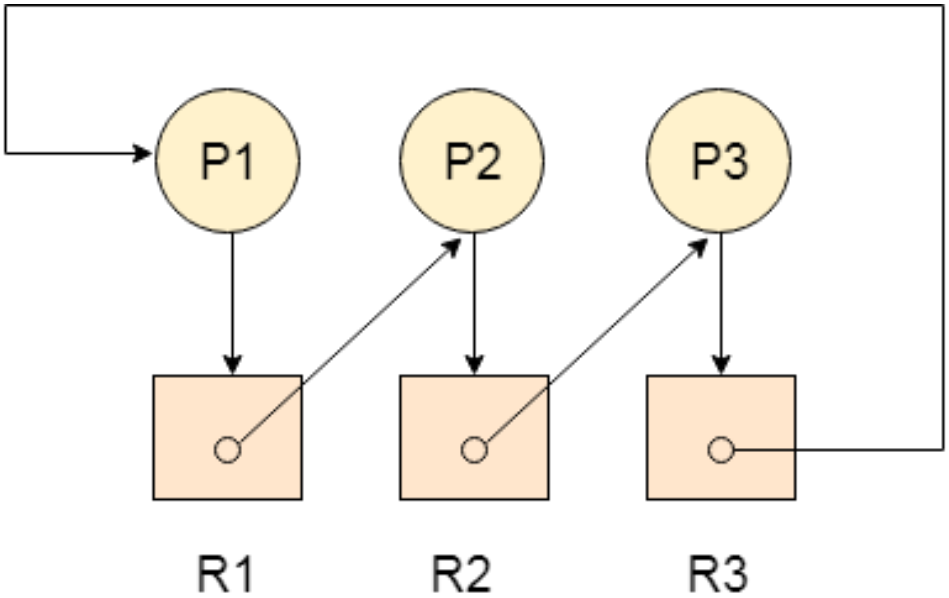
It gives the following information-

- There exist three processes in the system namely P1, P2 and P3.
- There exist two resources in the system namely R1 and R2.
- There exists a single instance of resource R1 and two instances of resource R2.
- Process P1 holds one instance of resource R1 and is waiting for an instance of resource R2.
- Process P2 holds one instance of resource R2 and is waiting for an instance of resource R1.
- Process P3 holds one instance of resource R2 and is not waiting for anything.



Deadlock Detection using RAG

- If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked.
- In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.
- The following example contains three processes P1, P2, P3 and three resources R2, R2, R3. All the resources are having single instances each.



- If we analyze the graph then we can find out that there is a cycle formed in the graph since the system is satisfying all the four conditions of deadlock.

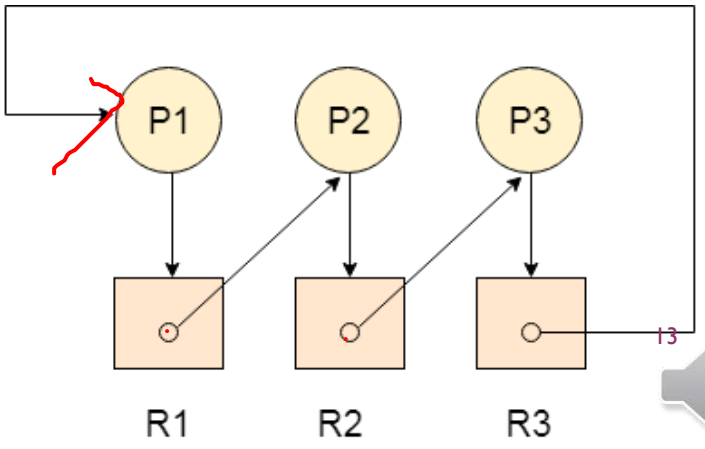
Allocation Matrix

Allocation matrix can be formed by using the Resource allocation graph of a system.

In Allocation matrix, an entry will be made for each of the resource assigned.

For Example, in the following matrix, an entry is being made in front of P1 and below R3 since R3 is assigned to P1.

Process	R1	R2	R3
P1	0	0	1
P2	1	0	0
P3	0	1	0



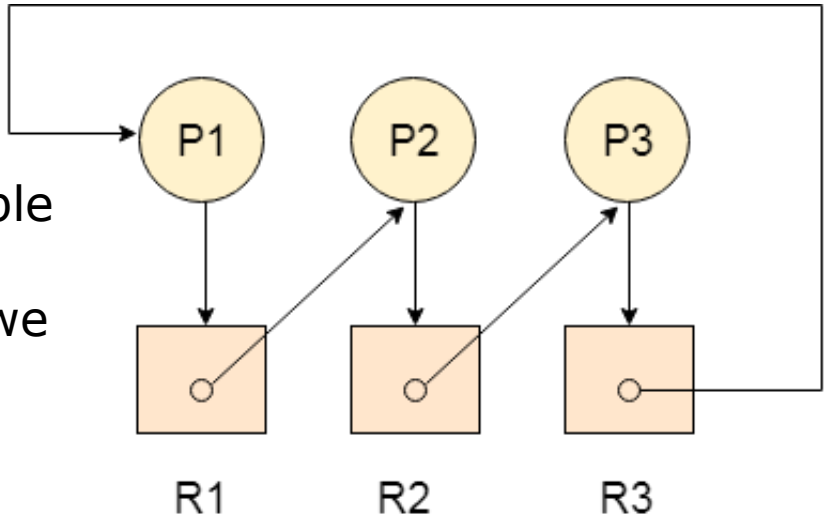
Request Matrix

In request matrix, an entry will be made for each of the resource requested. As in the following example, P1 needs R1 therefore an entry is being made in front of P1 and below R1.

Avial = (0,0,0)

Process	R1	R2	R3
P1	1	0	0
P2	0	1	0
P3	0	0	1

We cannot fulfil the demand of at least one process using the available resources therefore the system is deadlocked as determined, when we detected a cycle in the graph.



BASIC FACTS

- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$
  - if only one instance per resource type, then deadlock.
  - if several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks

- Deadlock Prevention.
  - Disallow one of the four necessary conditions for deadlock.
- Deadlock Avoidance.
  - Do not grant a resource request if this allocation have the potential to lead to a deadlock.

And recovery

- Deadlock Detection.
  - Always grant resource request when possible. Periodically check for deadlocks. If a deadlock exists, recover from it.

DEADLOCK PREVENTION

- Restrain the ways request can be made
- **Mutual Exclusion** – not required for sharable resources; must hold for non-sharable resources.
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.
- Require process to request and be allocated all its resources before it begins execution or allow process to request resources only when the process has none.
  - Two main disadvantages:
  - Low resource utilization; starvation possible.

DEADLOCK PREVENTION (CONT.)

- **No Preemption** –
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
  - Preempted resources are added to the list of resources for which the process is waiting.
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
- **Circular Wait** – impose a total ordering of all resource types and require that each process requests resources in an increasing order of enumeration.

This session overview:

Dead locks :

- Characterization
- Dead lock prevention

THANK YOU

Deadlock Avoidance, Deadlock Detection & Recovery

- Hamela K
- Department Of Computer Science
- Government First Grade College
- Malur 563 160

Deadlock Avoidance

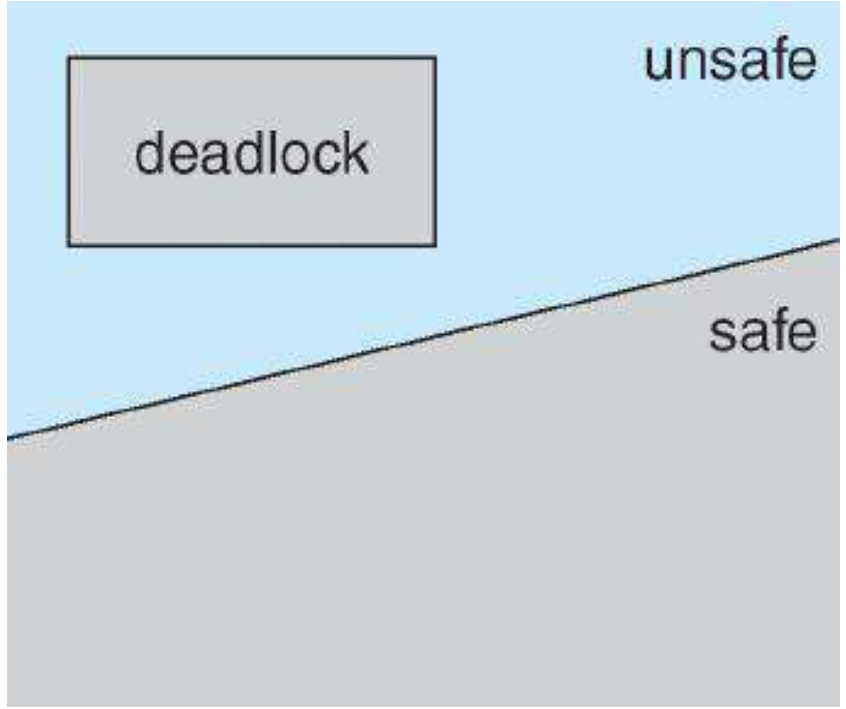
- To avoid deadlock the system has some additional priori information available:
  - Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
  - The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
  - Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes

Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in safe state if there exists a sequence <P1, P2, …, Pn> of ALL the processes in the systems such that for each Pi, the resources that Pi can still request can be satisfied by currently available resources + resources held by all the Pj, with j < I
- That is:
  - If Pi resource needs are not immediately available, then Pi can wait until all Pj have finished
  - When Pj is finished, Pi can obtain needed resources, execute, return allocated resources, and terminate
  - When Pi terminates, Pi +1 can obtain its needed resources, and so on

Basic Facts

- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ possibility of deadlock
- Avoidance ⇒ ensure that a system will never enter an unsafe state.

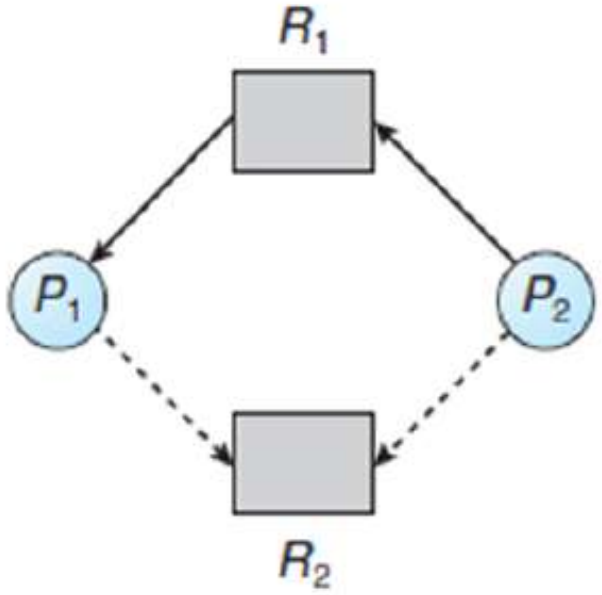


•Single instance of a resource type => Use a resource-allocation graph
•Multiple instances of a resource type => Use the banker's algorithm

RESOURCE ALLOCATION GRAPH FOR DEADLOCK AVOIDANCE

RESOURCE ALLOCATION GRAPH ALGORITHM.

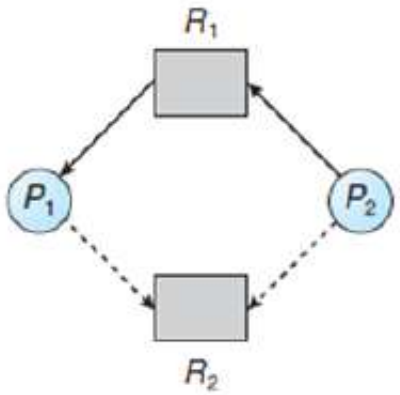
- A new edge CLAIM edge in addition to Assignment edge & Request edge.
  Pi → *Rj* indicates that process *Pi* may request resource *Rj* in future (------).
- When actual request is made, claim edge is converted into request edge.
- When resource is released, assignment edge is converted into claim edge.



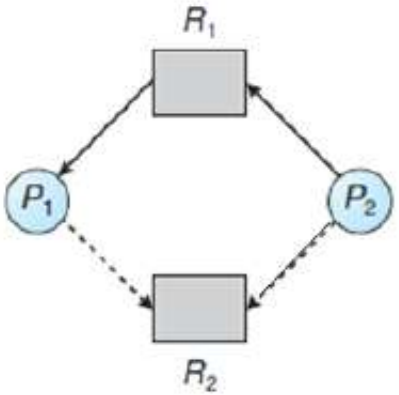
- Ensure there is no cycle in the graph.
- If no cycle exists, resource allocation will lead to a safe state.
- If cycle exists, resource allocation will lead to an unsafe state.

EXAMPLE

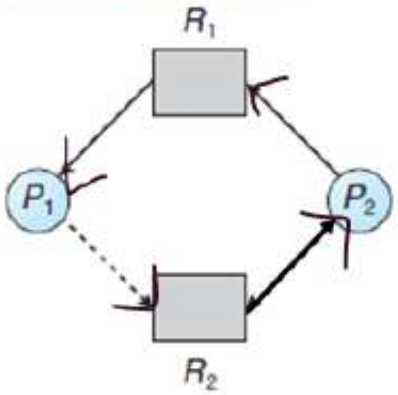
- P1 has R1 & P2 has requested for R1.



- P2 requests for R1



- It cannot be granted since it leads to this cycle in the graph.



Banker's Algorithm

- Banker's algorithm is a deadlock avoidance algorithm. It is named so because this algorithm is used in banking systems to determine whether a loan can be granted or not.

- Banker's algorithm works in a similar way in computers. Whenever a new process is created, it must exactly specify the maximum instances of each resource type that it needs.

Let us assume that there are **n** processes and **m** resource types. Some data structures are used to implement the banker's algorithm. They are:

- **Available:** It is an array of length **m**. It represents the number of available resources of each type. If **Available[j] = k**, then there are **k** instances available, of resource type **Rj**.
- **Max:** It is an **n x m** matrix which represents the maximum number of instances of each resource that a process can request. If **Max[i][j] = k**, then the process **Pi** can request atmost **k** instances of resource type **Rj**.
- **Allocation:** It is an **n x m** matrix which represents the number of resources of each type currently allocated to each process.
If **Allocation[i][j] = k**, then process **Pi** is currently allocated **k** instances of resource type **Rj**.
- **Need:** It is an **n x m** matrix which indicates the remaining resource needs of each process. If **Need[i][j] = k**, then process **Pi** may need **k** more instances of resource type **Rj** to complete its task.Need[i][j] = Max[i][j] - Allocation [i][j]

Bankers’s Algorithm

- Safety Algorithm
- Resource Request Algorithm

Safety Algorithm:

1. Let Work and Finish be vectors of length **m** and **n**, respectively. Initially,

```
Work = Available
Finish[i] =false for i = 0, 1, ... , n - 1.
```

This means, initially, no process has finished and the number of available resources is represented by the **Available** array.

2. Find an index **i** such that both

```
Finish[i] ==false
Needi <= Work
```

If there is no such i present, then proceed to step 4.

It means, we need to find an unfinished process whose need can be satisfied by the available resources. If no such process exists, just go to step 4.

3. Perform the following:

```
Work = Work + Allocation;
Finish[i] = true;
```

Go to step 2.

When an unfinished process is found, then the resources are allocated and the process is marked finished. And then, the loop is repeated to check the same for all other processes.

4. If **Finish[i]** == **true** for all i, then the system is in a safe state.

That means if all processes are finished, then the system is in safe state.

Resource Request Algorithm:

This describes the behavior of the system when a process makes a resource request in the form of a request matrix. The steps are:

1. If number of requested instances of each resource is less than the need (which was declared previously by the process), go to step 2.
2. If number of requested instances of each resource type is less than the available resources of each type, go to step 3. If not, the process has to wait because sufficient resources are not available yet.
3. Now, assume that the resources have been allocated. Accordingly do,

```
Available = Available - Requesti
Allocationi = Allocationi + Requesti
Needi = Needi - Requesti
```

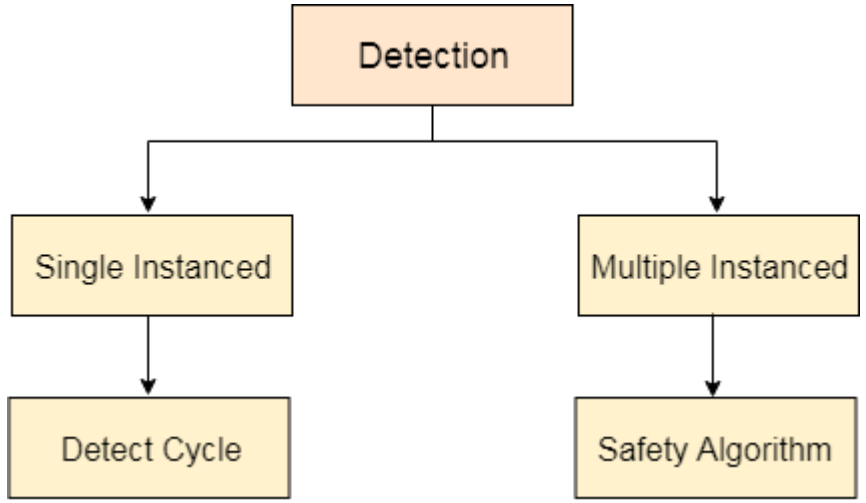
This step is done because the system needs to assume that resources have been allocated. So there will be less resources available after allocation. The number of allocated instances will increase. The need of the resources by the process will reduce. That's what is represented by the above three operations.

After completing the above three steps, check if the system is in safe state by applying the safety algorithm. If it is in safe state, proceed to allocate the requested resources. Else, the process has to wait longer.

Deadlock Detection

In this approach, The OS doesn't apply any mechanism to avoid or prevent the deadlocks. Therefore, the system considers that the deadlock will definitely occur. In order to get rid of deadlocks, The OS periodically checks the system for any deadlock. In case, it finds any of the deadlock then the OS will recover the system using some recovery techniques.

The main task of the OS is detecting the deadlocks. The OS can detect the deadlocks with the help of Resource allocation graph.



In single instanced resource types, if a cycle is being formed in the system then there will definitely be a deadlock.

On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough. We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.

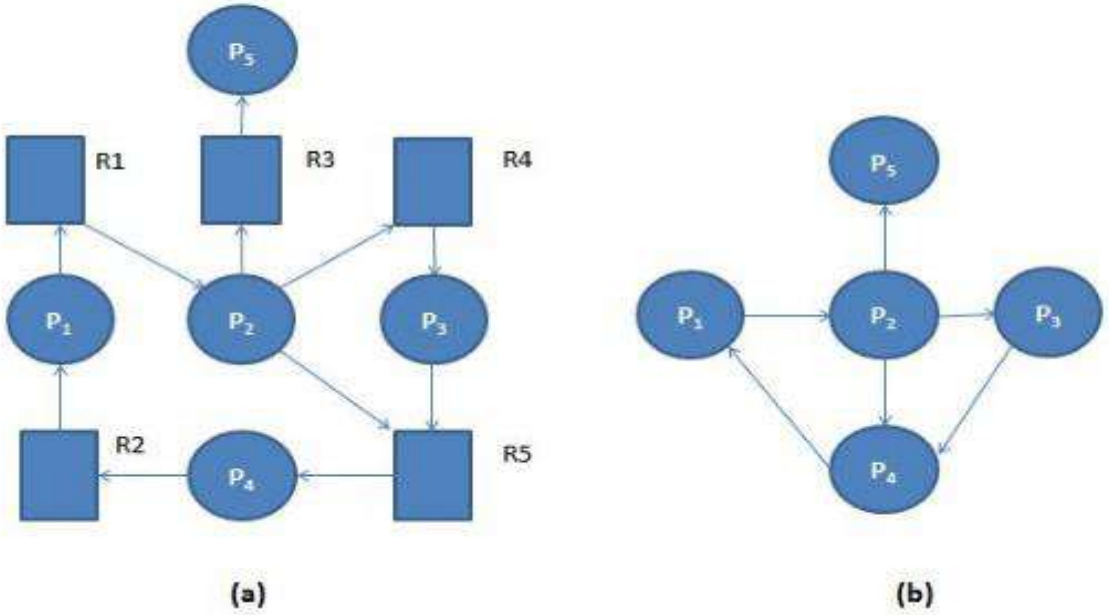
In general term, deadlock detection refers to:

Allow system to enter deadlock state
Detection algorithm
Recovery scheme

Single Instance of Each Resource Type

- Maintain wait-for graph
  - Nodes are processes
  - Pi → Pj if Pi is waiting for Pj
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n2 operations, where n is the number of vertices in the graph

Example: (a). 'Resource-Allocation Graph' and Corresponding (b). 'Wait-for Graph'



Several Instances of a Resource Type

- **Available**: A vector of length m indicates the number of available resources of each type.
- **Allocation**: An n x m matrix defines the number of resources of each type currently allocated to each process.
- **Request**: An n x m matrix indicates the current request of each process. If Request [i][j] = k, then process Pi is requesting k more instances of resource type.Rj.

Detection Algorithm

```
1. Let Work and Finish be vectors of length m and n, respectively Initialize:
    (a) Work = Available
    (b) For i = 1,2, …, n, if Allocationi ≠ 0, then Finish[i] = false;
     otherwise, Finish[i] = true

2. Find an index i such that both:
    (a) Finish[i] == false
    (b) Requesti ≤ Work
    If no such i exists, go to step 4

3. Work = Work + Allocationi Finish[i] = true go to step 2
```

4. If Finish[i] == false, for some i, 1 ≤ i ≤ n, then the system is in deadlock state. Moreover, if Finish[i] == false, then Pi is deadlocked

Recovery from Deadlock:

1. Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
    - Priority of the process
    - How long process has computed, and how much longer to completion
    - Resources the process has used
    - Resources process needs to complete
    - How many processes will need to be terminated
    - Is process interactive or batch?

2. Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor

University Question regarding Deadlock

Section A

- State the necessary condition for the deadlock to occur

Section A

- Explain Banker’s Algorithm
- Explain deadlock avoidance with example
- What is deadlock? Explain the characteristic of deadlock situation
- Explain any two methods of breaking and recovery from deadlock?

Thank you